Training Artificial Neural Network Architectures on the ANI-1 Dataset for Molecular Energy Prediction

1. Introduction

Machine Learning (ML), an ever evolving field focused on algorithms giving computers the ability to learn from the copious amounts of data accumulating over time and use inference to predict and make decisions without being explicitly programmed. ML is widely applicable across a multitude of fields including but not limited to healthcare, politics, and finance. This report focuses on the application of ML to chemistry. A key advantage of leveraging ML in chemistry is through its ability to accelerate research and allow for accurate prediction of complex molecular properties. One notable example of this application includes the most recent Chemistry Nobel Prize which focused on developing a Protein Language Model to predict protein structure[1].

A promising application of ML in chemistry is the prediction of potential energy surfaces (PES) for small molecules. Predicting energies accurately allows for deeper understanding of molecular stability, conformational dynamics and reaction pathways. Behler and Parrinello (2007) constructed neural network potentials (NNPs) that respect the physical symmetries of molecular systems via atom-centred symmetry functions[2]. Smith et al. (2017) built on this further and introduced the ANI-1 dataset and model[3] where they trained separate neural networks for hydrogen, carbon, nitrogen and oxygen on a large dataset of quantum mechanical (QM) DFT energies of organic molecules.

This work focuses on reproducing and further extending the work of Smith et al. with the ANI-1 dataset using TorchANI (Gao et al., 2020) and PyTorch (Paszke et al., 2019). Each atom's local chemical environment is encoded as an atomic environment vector (AEV) of length 384, which is computed using radial and angular symmetry functions. For each of the four elements, an artificial neural network (ANN) is trained, where the contributions of each of their energies yield a total molecular energy prediction. Whilst keeping this method constant, I focus on exploring the effect of different architectures, regularization strategies and hyperparameter choices on model performance and generalization using mean absolute error (MAE) in kcal/mol.

2. Methods

Dataset. The ANI-1 dataset was used for all experiments (Smith et al., 2017). As briefly mentioned in the introduction, the dataset contains DFT energies for small organic

molecules composed of H, C, N and O, sourced from the GDB-11 database. The dataset was split into training, validation and test sets with a 0.7, 0.2, 0.1 splitting strategy. The final model additionally uses normalization, using statistics calculated from the train set only to prevent data leakage.

Model Architecture. Each of the ANNs for each of the elements takes in a 384-dimensional input layer or AEV. The networks use these inputs to predict a scalar energy contribution. The total molecular energy of each molecule is calculated by taking a sum of the energy contributions of all atoms. After exploring several different architectures (outlined in the Results section), the final network employs one hidden layer of 128 dimensions with ReLU activation giving a total of 197,636 parameters across all four networks for the four atoms.

Training and Evaluation. All models were trained using the Adam optimizer[6] with a learning rate of 1e-3 and L2 regularization with penalty term 1e-7. A batch size of 8,192 molecules were used and training was conducted for 30 epochs with early stopping. Training was performed on 1 GTX 2080 Ti GPU. Model performance was calculated using mean absolute error (MAE) in kcal/mol, with energies predicted in Hartree units and converted using a factor of 1 Hartree to 627.509 kcal/mol. To assess the robustness of the model, I performed 5 independent training runs and 5-fold cross-validation using normalization.

3. Results and Discussion

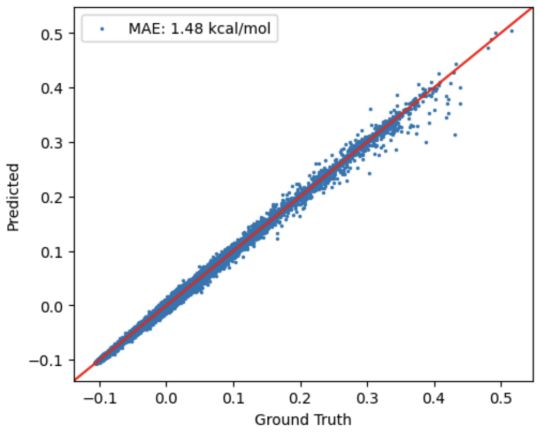
The table below shows the different architectures with different hyperparameters that I used to find the optimal architecture for the goal of this report. All models used the same train/validation/test split.

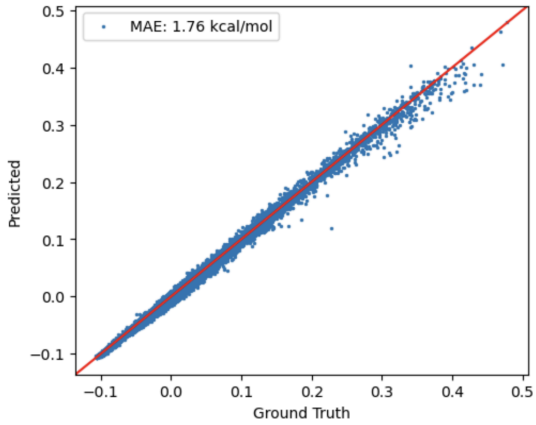
Layers	Learning Rate	Epochs	L2	Dropouts	Activation Function	Train MAE (kcal/mol)	Validation MAE (kcal/mol)	Test MAE (kcal/mol)
[384, 128, 1]	1e-3	30	0	0.0	ReLU	1.47	1.49	1.48
[384, 128, 1]	1e-3	30	1e-6	0.0	ReLU	1.75	1.75	1.76
[384, 128, 1]	1e-3	30	1e-7	0.0	ReLU	1.42	1.43	1.43
[384, 128, 1]	1e-3	30	1e-8	0.0	ReLU	1.71	1.73	1.73
[384, 128, 1]	1e-3	50	1e-7	0.1	ReLU	3.60	3.61	3.62
[384, 256, 128, 1]	1e-3	50	1e-7	0.1	ReLU	1.80	1.80	1.81

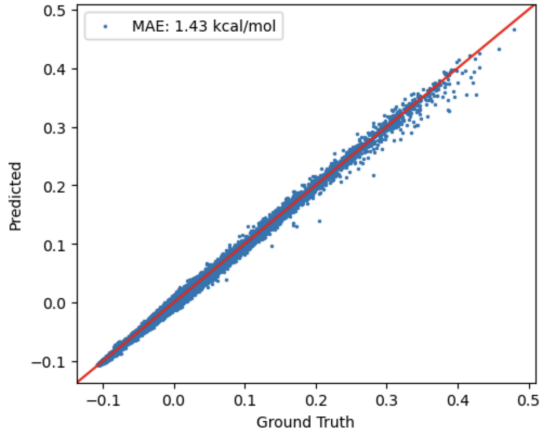
[384, 200, 150, 80, 1]	1e-3	70	1e-7	0.1	ReLU	2.30	2.30	2.31
[384, 200, 150, 80, 1]	1e-3	90	1e-7	0.1	ReLU	1.63	1.63	1.64

Figure 1. Table summary of all hyperparameters and architectures with corresponding results on train, validation and test sets.

First, I began by investigating the effect of different L2 regularization weights on the architecture version with 1 hidden layer. From my results above, it is evident that using an L2 weight of 1e-7 was optimal, which is why I then continued to use it in the subsequent runs. Both higher and smaller weights (1e-6 and 1e-8) led to higher errors above 1.70 kcal/mol. This suggests that 1e-7 meets the optimum tradeoff between underfitting and overfitting in terms of bias and variance for this architecture. The figures below show how the test errors change across the different L2 regularization weights.







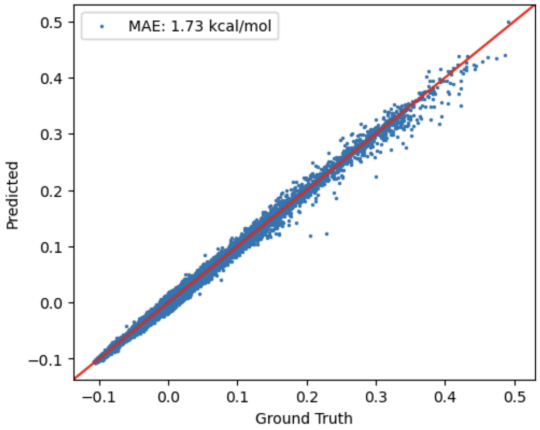


Figure 2. Predicted vs ground truth energies for the test error across the four L2 regularization strategies: no regularization (top left), L2 = 1e-6 (top right), L2 = 1e-7 (bottom left), L2 = 1e-8 (bottom right). The red line indicates a perfect prediction. L2 = 1e-7 has tightest scatter around the perfect predictor line.

In terms of dropout, the p=0.1 addition surprisingly hurt model performance more than expected, with the MAE rising to 3.62 kcal/mol for the test error even after an increased number of epochs (50) of training as shown by the figures below. This suggests that dropout doesn't work too well on this simpler architecture, consequently leading me to test dropout on deeper network architectures.

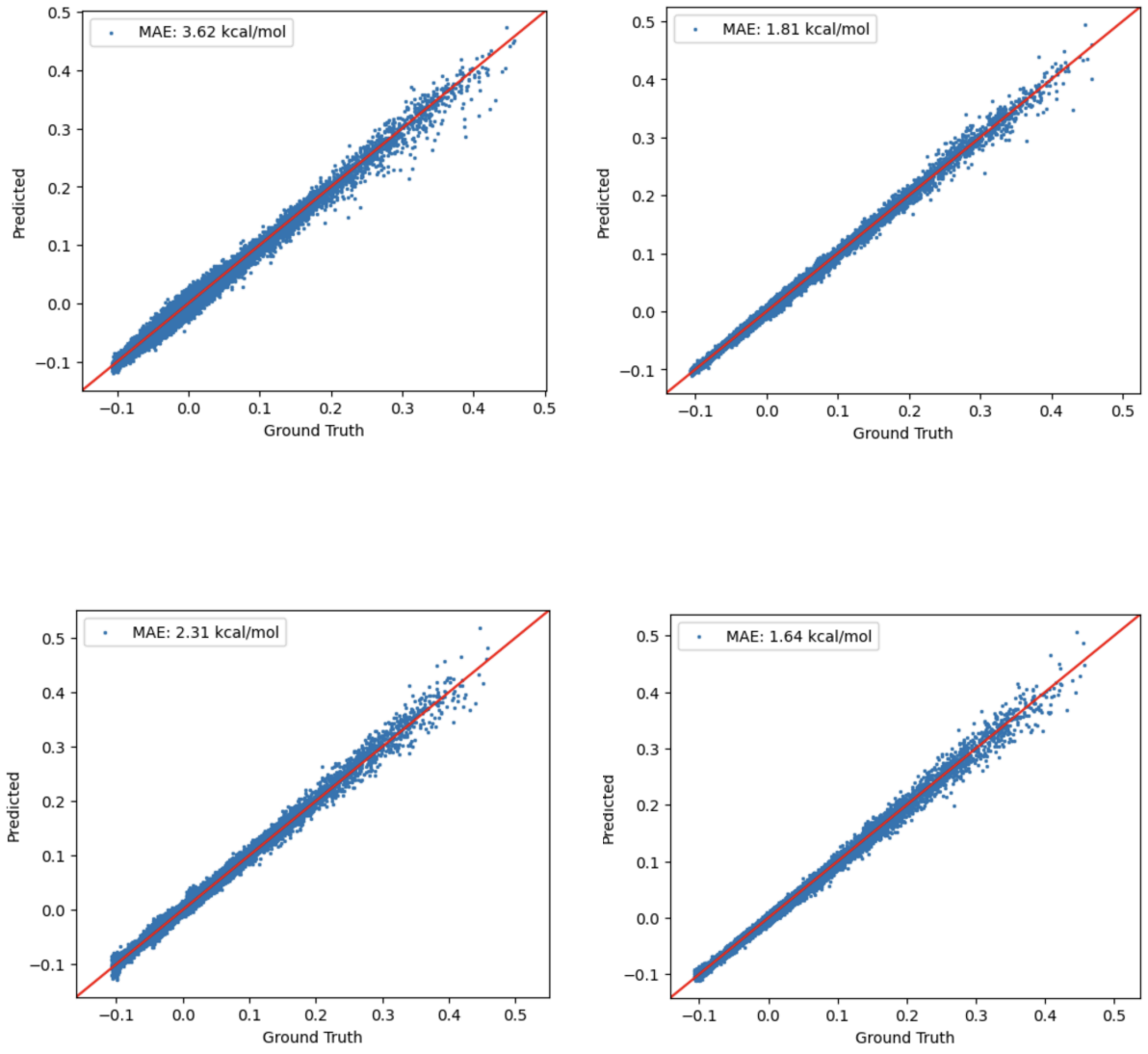


Figure 3. Predicted vs ground truth energies for the test error across different architectures with dropout p=0.1. Architecture [384, 128, 1], dropout p=0.1 (top left), architecture [384, 256, 128, 1], dropout p=0.1 (top right), architecture [384, 200, 150, 80, 1], dropout p=0.1 with 70 epochs (bottom left) and same architecture with 90 epochs (bottom right). The red line indicates a perfect prediction. Dropout with 3

hidden layers for 90 epochs performs best out of these options, but still not as well as the simpler 1 hidden layer model with no dropout.

However, deeper architectures still did not improve performance compared to those of the single hidden layer runs. The deeper architectures with dropout reached 1.81 kcal/mol and 1.64 kcal/mol for 2 hidden layers and 3 hidden layers respectively. From this, I deduced that the simple scale of using 4 atoms (H, C, N and O) does not require a complex model, and that a single hidden layer is sufficient to capture the relevant relationships from the ANI-1 dataset to predict molecular energies.

Once the final architecture (1 hidden layer, [384, 128, 1], no dropout, L2=1e-7) was selected, I normalized the data using training data statistics and ran 5-fold cross validation and 5 full independent runs. The results of this confirmed that the final architecture of this model is suitable for the task at hand. For cross validation, the model achieved a mean MAE of 1.073 kcal/mol with a standard deviation of 0.041 kcal/mol across the 5 folds, demonstrating low variance and strong generalisation, since the model is not sensitive to the choice of training data split. For the multiple runs, the 5 independent runs resulted in a mean test MAE of 1.197 kcal/mol with standard deviation 0.077 kcal/mol, again indicating good overall generalisation.

Overall, these results are comparable to the result reported by Smith et al. (2017). More notably, these results were achieved using a simpler architecture and fewer parameters than the original ANI-1 model.

4. Conclusion

In conclusion, my experiments have shown that complex models do not always perform better. Through systematic exploration of architecture depth, different regularization strategies (L2 weights) and dropout, it is clear from the results that the simpler architecture with 1 hidden layer and L2 weight outperformed the other models I ran. There are limitations to take into account however. This current model only uses 4 atoms (H, C, N, O) so the conclusion might different when larger molecules with a more diverse range of atoms are considered. Additionally, there are more model architectures and hyperparameters that I could have experimented with, such as the activation function or the learning rate.

On a larger scale, whilst my work is based on the ANI-1 dataset and model in the paper by Smith et al. (2017), it still motivates the power and potential of leveraging ML tools and techniques within the realm of chemistry. Future work could explore larger subsets of the ANI-1 dataset, employ more thorough hyperparameter tuning strategies and perform longer training schedules. However, if a simple architecture with L2

regularization is able to predict molecular energies with an average MAE of ~1 kcal/mol, there is significant promise for more sophisticated models to generalize well to more complex and diverse sets of molecules.

5. References

1. Jumper, J., Evans, R., Pritzel, A., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, *596*, 583–589. https://doi.org/10.1038/s41586-021-03819-2
2. Behler, J., & Parrinello, M. (2007). Generalized neural-network representation of high-dimensional potential-energy surfaces. Physical Review Letters, 98(14), 146401. https://doi.org/10.1103/PhysRevLett.98.146401
3. Smith, J. S., Isayev, O., & Roitberg, A. E. (2017). ANI-1: an extensible neural network potential with DFT accuracy at force field computational cost. Chemical science, 8(4), 3192–3203. https://doi.org/10.1039/c6sc05720a
4. Gao, X., Ramezanghorbani, F., Isayev, O., Smith, J. S., & Roitberg, A. E. (2020). TorchANI: A free and open source PyTorch-based deep learning implementation of the ANI neural network potentials. Journal of Chemical Information and Modeling, 60(7), 3408–3415. https://doi.org/10.1021/acs.jcim.0c00451
5. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32. https://arxiv.org/abs/1912.01703
6. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. Proceedings of the 3rd International Conference on Learning Representations (ICLR). https://doi.org/10.48550/arXiv.1412.6980